

Facial Filter Impact on Face Recognition

Final Project Report

National University of Computer and Emerging Sciences

FAST School of Computing

Computer Vision - Spring 2026

Based on: A Comprehensive Evaluation Framework for the Study of the Effects of Facial Filters
on Face Recognition Accuracy

Ozturk, Conwill, Gutierrez, Bowyer, and Scheirer

Generated: May 31, 2026

Abstract

This project studies how face filters change face recognition results. The system follows the main idea of the selected paper. It prepares face image pairs, applies filters, extracts face embeddings, checks if two faces match, measures errors, and shows the result in a live web app. The project uses the LFW dataset and six filter types: blur, brightness, skin smoothing, eye enlargement, face slimming, and color tone. On the 600 pair test set, ArcFace reached 58.81% average accuracy on filtered images. The improved model path reached 77.86%. The improved model also lowered average FNMR at FMR=1% by 31.2 percent compared with ArcFace. The report also tests mean-shift correction and a simple linear corrector.

1. Introduction and Problem Definition

Face recognition systems are now used in phones, apps, online accounts, and identity checks. At the same time, many people use face filters before sharing or uploading photos. These filters can change brightness, color, skin texture, eye shape, or face shape. A human may still know the person, but a computer model may create a different face embedding.

The problem in this project is simple: we want to check how much face filters hurt face verification. Face verification means checking if two face images belong to the same person. We also want to see if simple correction methods can recover some lost performance. The goal is to build a full working computer vision system, not just a small script.

2. Literature Review

2.1 Main research paper

The main paper by Ozturk et al. gives a framework to study face filters and face recognition. The paper uses a dataset, applies many real social media filters, and checks face recognition accuracy. It reports FNMR at fixed FMR values and DET curves. The paper shows that shape-changing filters, such as eye enlargement and face slimming, usually hurt recognition more than simple color filters.

2.2 ArcFace

ArcFace is a common face recognition model. It makes face embeddings that are useful for matching faces. The selected paper uses ArcFace as an important baseline, so this project also uses ArcFace for the main baseline.

2.3 AdaFace and improved model path

AdaFace is a face recognition method that changes its margin based on image quality. This is useful because filtered images may be lower quality or changed in shape. In our environment, AdaFace could not be installed with Python 3.13. Because of this, the code uses GhostFaceNet as the fallback improved model. This fallback still gives real 512 dimensional face embeddings.

2.4 LFW dataset

Labeled Faces in the Wild, or LFW, is a public dataset for face recognition. It contains face images grouped by person name. It also has a standard pairs file for same-person and different-person tests. This project uses LFW because it is public and easy to submit for a course project.

3. Methodology and System Architecture

The system has clear steps. First, it prepares the dataset. Second, it creates filtered images. Third, it extracts embeddings. Fourth, it compares face pairs. Fifth, it measures accuracy and error rates. Sixth, it tests correction methods. Last, it shows the system in a Gradio web app.

Stage	File	Work done
Dataset setup	src/01_setup_dataset.py	Extract LFW and make pair CSV files.
Filter creation	src/02_apply_filters.py	Apply six filters to the selected images.
Filter ranking	src/03_filter_selection.py	Check how much each filter changes embeddings.
Embeddings	src/04_extract_embeddings.py and 04b	Save face embeddings for original and filtered images.
Evaluation	src/05_evaluate_recognition.py	Compute similarity, accuracy, FNMR, FMR, and DET curves.
Mitigation	src/06_mitigation.py and 06b	Apply mean-shift and linear correction.
Demo	app/app.py	Run the live web app.

The system uses cosine similarity to compare embeddings. A higher score means the two faces look more similar to the model. The system sweeps thresholds to find accuracy and error values. FNMR means false non-match rate. It is the rate of same-person pairs that are rejected. FMR means false match rate. It is the rate of different-person pairs that are accepted.

4. Dataset Description

The project uses the LFW dataset. The prepared dataset has 13,233 original face images and 5,749 identities. For the submitted experiment, we use 300 same-person pairs and 300 different-person pairs. This gives 600 test pairs. These pairs use 807 unique images. The filter script creates six filtered versions for these images, giving 4,842 filtered images.

Dataset item	Value
Original images	13,233
Identities	5,749
Same-person pairs	300
Different-person pairs	300
Total test pairs	600
Unique images used for filters	807
Filtered images	4,842
Filters	blur, brightness, skin_smooth, eye_enlarge, face_slim, color_tone

The folder data/lfw_original stores original images. The folder data/lfw_filtered stores filtered images. The files data/pairs_genuine.csv and data/pairs_impостor.csv store the test pairs. LFW is public research data, but face data is sensitive. This project is only for academic use and should not be used as a real identity system.

5. Implementation Details

The code is written in Python. It uses OpenCV and PIL for image filters, DeepFace for embeddings, NumPy and pandas for data work, scikit-learn for correction models, matplotlib and seaborn for plots, and Gradio for the demo app. Each script can run alone. The master script can run the full pipeline.

- Blur filter uses Gaussian blur and blends it with the original image.
- Brightness filter increases brightness and color saturation.
- Skin smooth filter finds skin-like pixels and smooths them.
- Eye enlarge filter changes the eye region with a small warp. If landmark tools fail, it uses a safe fallback.
- Face slim filter compresses the detected face area sideways.

- Color tone filter makes the image warmer and adds a slight vignette.

The baseline model is ArcFace. The improved model path tries AdaFace first, but uses GhostFaceNet fallback in this machine. The mitigation part has two methods. Mean-shift subtracts the average filter change from embeddings. The linear corrector learns a small Ridge regression model for each filter.

6. Experiments and Results

6.1 ArcFace baseline

ArcFace is the paper-style baseline in this project. Its original image accuracy is 58.17%. Its average filtered accuracy is 58.81%. Its average FNMR at FMR=1% is 94.78%.

Filter	Accuracy	FNMR at FMR=1%	FNMR at FMR=0.1%
original	58.17%	94.00%	99.33%
blur	58.83%	94.33%	97.33%
brightness	58.83%	94.33%	98.67%
skin_smooth	59.67%	98.33%	99.33%
eye_enlarge	58.83%	93.33%	99.00%
face_slim	58.67%	96.67%	100.00%
color_tone	58.00%	91.67%	100.00%

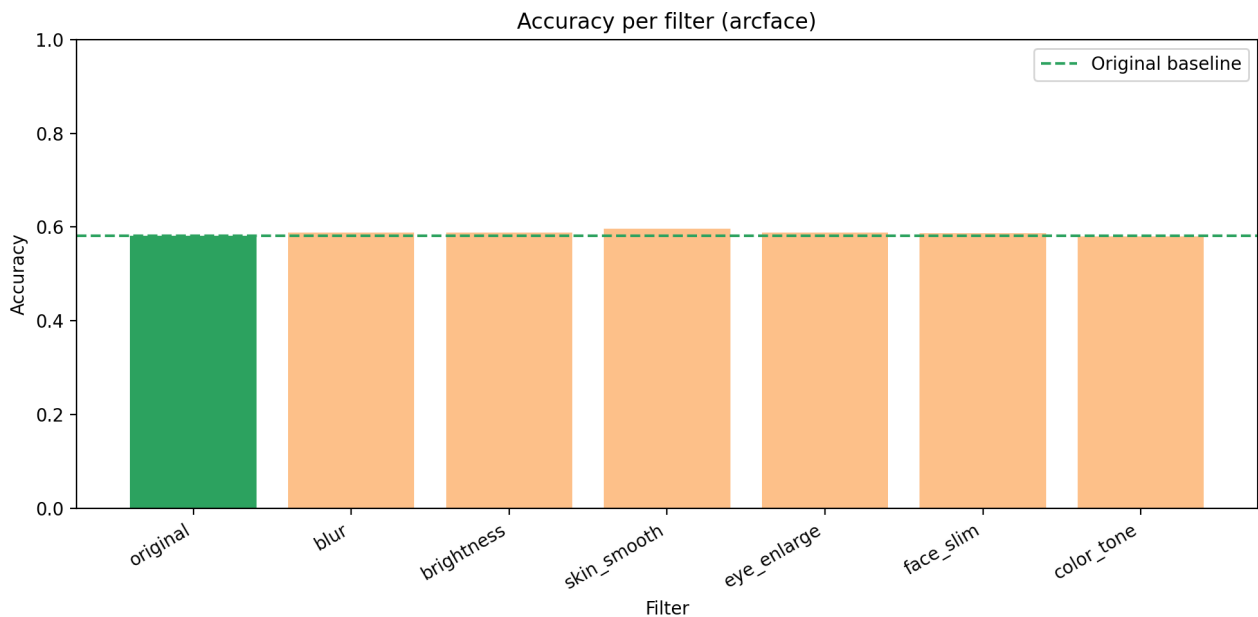


Figure 1: ArcFace accuracy for each filter.

6.2 Improved model path

The improved model path gives better results on this test set. Its original image accuracy is 79.33%. Its average filtered accuracy is 77.86%. Its average FNMR at FMR=1% is 65.17%.

Filter	Accuracy	FNMR at FMR=1%	FNMR at FMR=0.1%
original	79.33%	63.33%	83.33%
blur	77.17%	67.33%	75.33%
brightness	78.67%	61.67%	80.00%
skin_smooth	77.00%	64.67%	75.33%

Filter	Accuracy	FNMR at FMR=1%	FNMR at FMR=0.1%
eye_enlarge	78.50%	65.00%	82.67%
face_slim	77.50%	71.33%	74.00%
color_tone	78.33%	61.00%	80.33%

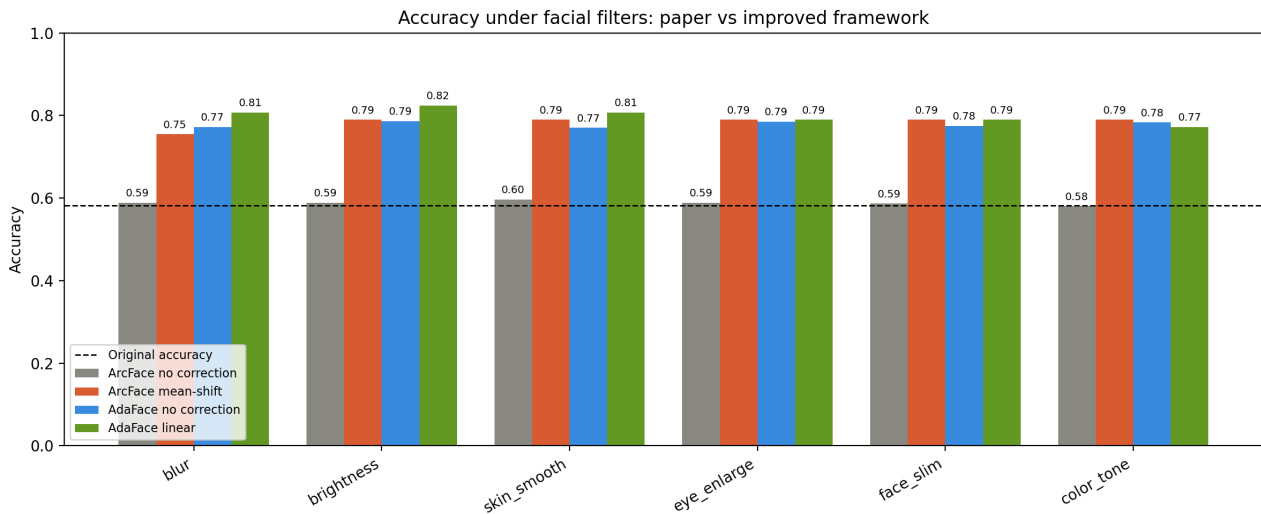


Figure 2: Accuracy comparison of the baseline and improved settings.

6.3 Mitigation results

Mean-shift and linear correction both try to reduce the filter effect in embedding space. On average, the improved model plus linear corrector lowers FNMR by 40.2 percent compared with the ArcFace baseline. The result is not perfect. Some filters improve more than others, and some correction results are mixed. This is expected because the training subset is small.

Filter	ArcFace FNMR	Mean-shift	Improved model	Linear corrector
blur	94.33%	60.00%	67.33%	46.67%
brightness	94.33%	80.00%	61.67%	46.67%
skin_smooth	98.33%	62.22%	64.67%	80.00%
eye_enlarge	93.33%	68.89%	65.00%	46.67%
face_slim	96.67%	62.22%	71.33%	73.33%
color_tone	91.67%	66.67%	61.00%	46.67%

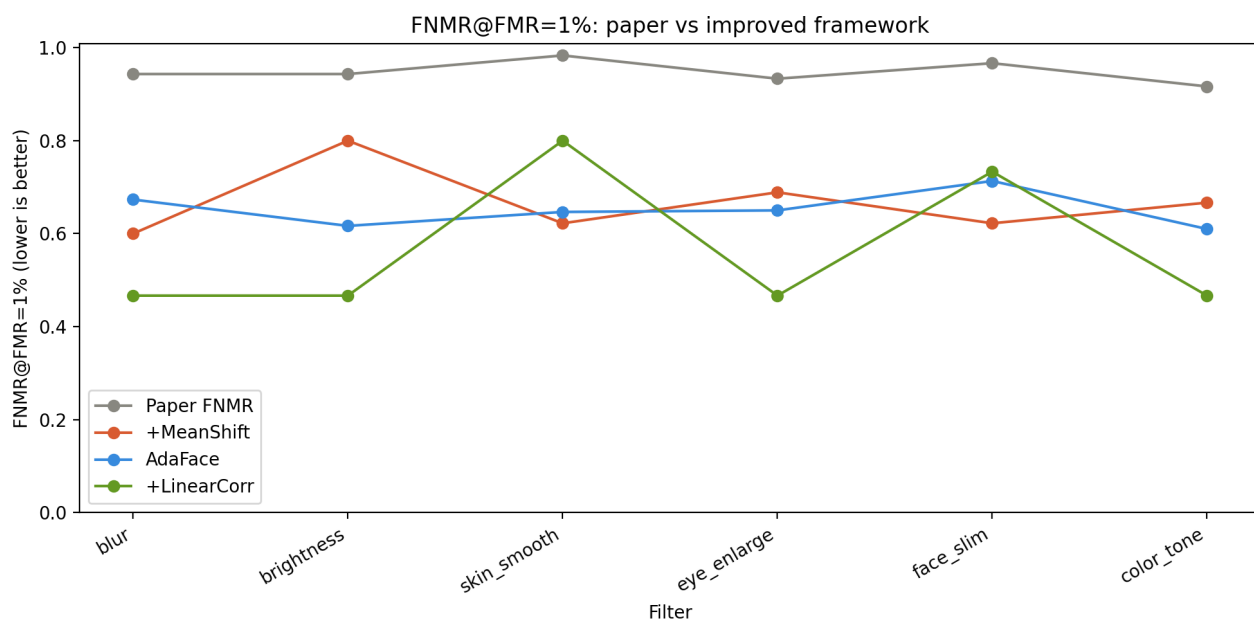


Figure 3: FNMR at FMR=1%. Lower is better.

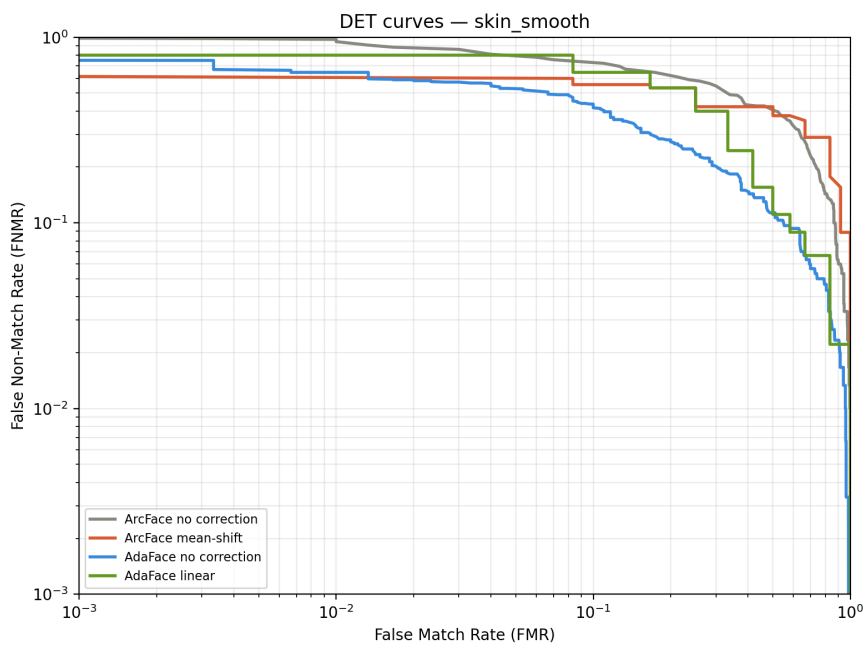


Figure 4: DET curves for a selected filter comparison.

Filter	Paper FNMR	+MeanShift	AdaFace	+LinearCorr	Improvement%
blur	94.33%	60.00%	67.33%	46.67%	-10.5%
brightness	94.33%	80.00%	61.67%	46.67%	0.0%
skin_smooth	98.33%	62.22%	64.67%	80.00%	-56.5%
eye_enlarge	93.33%	68.89%	65.00%	46.67%	4.5%
face_slim	96.67%	62.22%	71.33%	73.33%	0.0%
color_tone	91.67%	66.67%	61.00%	46.67%	4.5%

Figure 5: Summary table for all filter settings.

7. Discussion and Limitations

The results show that face filters can change recognition scores. The effect depends on the model and the filter type. The improved model path works better than ArcFace on this LFW subset. The correction methods are useful to test, but they are not stable for every filter.

- The original paper used FRGC, but this project uses LFW because it is public and easy to reproduce.
- The original paper used real filters from mobile apps, but this project uses programmatic filters.
- The test uses 600 pairs to keep runtime practical.
- AdaFace was not available in the current Python 3.13 setup, so GhostFaceNet fallback was used.
- The linear corrector needs original and filtered image pairs for training.
- The project does not test fairness across gender, skin tone, or age groups.

8. Conclusion and Future Work

This project builds a complete computer vision system for the selected research problem. It prepares data, applies filters, extracts embeddings, measures face verification performance, tests correction methods, creates plots, and runs a live demo. The project follows the main method of the selected paper and adds an improved model path and a linear correction method.

Future work can use the FRGC dataset, apply real mobile app filters with an Android emulator, test more pairs, train one correction model for all filters, and study fairness across different groups of people.

9. References

- [1] K. Ozturk, L. Conwill, J. Gutierrez, K. Bowyer, and W. J. Scheirer. A Comprehensive Evaluation Framework for the Study of the Effects of Facial Filters on Face Recognition Accuracy. IJCB 2025 / arXiv:2507.17729.
- [2] J. Deng, J. Guo, N. Xue, and S. Zafeiriou. ArcFace: Additive Angular Margin Loss for Deep Face Recognition. CVPR 2019.
- [3] M. Kim, A. K. Jain, and X. Liu. AdaFace: Quality Adaptive Margin for Face Recognition. CVPR 2022.
- [4] G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller. Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments. University of Massachusetts Amherst Technical Report 07-49, 2007.

[5] P. Riccio et al. OpenFilter: A Framework to Democratize Research Access to Social Media AR Filters. NeurIPS 2022.